```
// functions/api/checkout.js  —  Cloudflare Pages Function for Vintage of 4 Seaso
// It answers at the web address  /api/checkout
//
// The website sends the CART (product names, colors/sizes, quantities) — NEVER p
// This file looks up the real prices below, so a customer can't change what they
// then asks Stripe for a hosted checkout page and sends the customer there.
// No extra libraries needed: it talks to Stripe's web API directly.
//
// Settings (Cloudflare dashboard > your Pages project > Settings > Variables and
//   STRIPE_SECRET_KEY  your Stripe secret key (starts with sk_test_ or sk_live_)
//   SITE_URL           your real site address, starting with https://

// ---------------------------------------------------------------
// PRICE LIST (US dollars). When Luso changes a price, change it here AND on the
// ---------------------------------------------------------------
const CATALOG = {
  "Handcrafted Winter Angel": {"p":84.99},
  "Vintage Carousel Horse Drum Ornament": {"p":74.99},
  "Gold Leaf Teardrop Ornament": {"p":59.99},
  "Antique Carriage Ornament": {"p":124.99},
  "Guitar Ornament": {"p":79.99},
  "Chandelier Ornament": {"p":114.99},
  "High Heel Ornament": {"p":44.99},
  "Medallion Ornament": {"p":94.99},
  "Large Medallion Ornament": {"p":134.99},
  "Mother Mary Ornament": {"p":79.99},
  "Doves & Crown Ornament": {"p":79.99},
  "Throne Ornament": {"p":74.99},
  "Peacock Ornament": {"p":114.99},
  "Ornament Gift Box": {"box":true},
  "Jeweled Bow Ornament": {"p":84.99},
  "Praying Mary Ornament": {"p":79.99},
  "Angel Ornament": {"p":84.99},
  "Sapphire Cameo Ornament": {"p":84.99},
  "Angel of Eternal Love Ornament": {"p":89.99},
  "Ballet Slippers Ornament": {"p":84.99},
  "Ballerina Ornament": {"p":84.99},
  "Masquerade Mask Ornament": {"p":84.99},
  "Horse Head Ornament": {"p":84.99},
  "Carousel Horse Medallion": {"p":74.99},
};

const has = (obj, key) => Object.prototype.hasOwnProperty.call(obj, key);
```

```
// Price for a product + variant. Variants look like "Red" or "Cream · Large";
// for products with sizes, the text after the last " · " picks the price.
function priceFor(name, variant) {
  if (!has(CATALOG, name)) return null;
  const item = CATALOG[name];
  if (item.sizes) {
    const size = String(variant).split(' · ').pop();
    return has(item.sizes, size) ? item.sizes[size] : null;
  }
  return item.p;
}

// Same ZIP-based shipping estimate the website shows. lineCount = number of diff
function shippingCost(lineCount, zip) {
  const base = 6, perExtra = 1.5, extra = (lineCount - 1) * perExtra;
  const digit = zip ? Number(zip[0]) : null;
  let zoneFee = 3;
  if (digit !== null) zoneFee = Math.round(Math.abs(9 - digit) * 0.9 * 10) / 10;
  return Math.round((base + extra + zoneFee) * 100) / 100;
}

const clean = (v, max) => (typeof v === 'string' ? v.replace(/[\u0000-\u001f]/g,

// Gift Box: the price is the ornaments chosen inside it (box included). 3 or mor
const BOX_FEE_CENTS = 0, BOX_FREE_AT = 3, BOX_MAX_ITEMS = 12, BOX_MAX_LINES = 8;
function boxQuote(items) {
  if (!Array.isArray(items) || items.length < 1 || items.length > BOX_MAX_LINES)
  let cents = BOX_FEE_CENTS, count = 0;
  const parts = [];
  for (const it of items) {
    const [n, c, q] = Array.isArray(it) ? it : [];
    const quantity = Number(q);
    if (typeof n !== 'string' || !has(CATALOG, n) || CATALOG[n].box || typeof CAT
    if (!Number.isInteger(quantity) || quantity < 1 || quantity > BOX_MAX_ITEMS)
    count += quantity;
    if (count > BOX_MAX_ITEMS) return null;
    cents += Math.round(CATALOG[n].p * 100) * quantity;
    parts.push(quantity + 'x ' + n.replace(/ Ornament$/, '') + ' (' + clean(typeo
  }
  return { cents, count, free: count >= BOX_FREE_AT, contents: parts.join(', ') }
}
const say = (message, status) =>
  new Response(message, { status, headers: { 'Content-Type': 'text/plain; charset

export async function onRequest(context) {
```

```
const { request, env } = context;

if (request.method !== 'GET' && request.method !== 'POST') return say('Method n
if (!env.STRIPE_SECRET_KEY) return say('Server misconfiguration: STRIPE_SECRET_
const siteUrl = env.SITE_URL;
if (!siteUrl || !/^https?:\/\//.test(siteUrl)) {
  return say('Server misconfiguration: SITE_URL is missing or must start with h
}

// Read the details (address bar for GET, JSON body for POST)
let q;
if (request.method === 'GET') {
  const p = new URL(request.url).searchParams;
  q = { cart: p.get('cart'), zip: p.get('zip'), email: p.get('email'), name: p.
} else {
  try { q = await request.json(); } catch (e) { return say('Invalid request bod
  if (!q || typeof q !== 'object') return say('Invalid request body.', 400);
}

let cart;
try { cart = typeof q.cart === 'string' ? JSON.parse(q.cart) : q.cart; } catch
if (!Array.isArray(cart) || cart.length < 1 || cart.length > 30) {
  return say('Your cart looks empty. Please go back and try again.', 400);
}

// Build the form Stripe expects (every price comes from CATALOG above)
const form = new URLSearchParams();
form.append('mode', 'payment');
let n = 0;
for (const row of cart) {
  const [name, variant, qty, boxItems] = Array.isArray(row) ? row : [];
  const quantity = Number(qty);
  let unitCents, label;
  if (typeof name === 'string' && has(CATALOG, name) && CATALOG[name].box) {
    const box = boxQuote(boxItems);
    if (box === null) return say('Your gift box could not be read. Please go ba
    unitCents = box.cents;
    label = clean(name, 60) + ' — ' + clean(String(variant), 30) + ' — contains
    label = label.length > 250 ? label.slice(0, 247) + '...' : label;
  } else {
    const price = typeof name === 'string' ? priceFor(name, variant) : null;
    if (typeof price !== 'number') return say('One of the items in your cart co
    unitCents = Math.round(price * 100);
    label = clean(name, 100) + ' — ' + clean(String(variant), 60);
  }
  if (!Number.isInteger(quantity) || quantity < 1 || quantity > 50) {
```

```
    return say('One of the items in your cart could not be found. Please go bac
  }
  form.append(`line_items[${n}][price_data][currency]`, 'usd');
  form.append(`line_items[${n}][price_data][product_data][name]`, label);
  form.append(`line_items[${n}][price_data][unit_amount]`, String(unitCents));
  form.append(`line_items[${n}][quantity]`, String(quantity));
  n++;
}

const zip = /^\d{5}$/.test(clean(q.zip, 5)) ? clean(q.zip, 5) : '';
const ship = shippingCost(cart.length, zip);
form.append(`line_items[${n}][price_data][currency]`, 'usd');
form.append(`line_items[${n}][price_data][product_data][name]`, 'Shipping (esti
form.append(`line_items[${n}][price_data][unit_amount]`, String(Math.round(ship
form.append(`line_items[${n}][quantity]`, '1');

const email = clean(q.email, 200);
if (/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) form.append('customer_email', ema
form.append('shipping_address_collection[allowed_countries][0]', 'US');
form.append('phone_number_collection[enabled]', 'true');
form.append('metadata[name]', clean(q.name, 200));
form.append('metadata[phone]', clean(q.phone, 40));
form.append('metadata[address]', clean(q.addr, 300));
form.append('metadata[zip]', zip);
form.append('success_url', siteUrl + (siteUrl.includes('?') ? '&' : '?') + 'pai
form.append('cancel_url', siteUrl);

// Ask Stripe for the hosted checkout page
let data = {};
let ok = false;
try {
  const resp = await fetch('https://api.stripe.com/v1/checkout/sessions', {
    method: 'POST',
    headers: { Authorization: 'Bearer ' + env.STRIPE_SECRET_KEY, 'Content-Type'
    body: form.toString(),
  });
  data = await resp.json().catch(() => ({}));
  ok = resp.ok && !!data.url;
  if (!ok) console.error('Stripe error', resp.status, data && data.error && dat
} catch (e) {
  console.error('Could not reach Stripe', e && e.message);
}
if (!ok) return say('Something went wrong starting checkout. Please go back and

if (request.method === 'POST') {
  return new Response(JSON.stringify({ url: data.url }), { status: 200, headers
```

```
  }
  return new Response(null, { status: 303, headers: { Location: data.url } });
}
```